Pilotwings 64 (US): ROM and RAM layout

Before = the original game (8 MB ROM, console without Expansion Pak). After = a hack ROM of Hawk's Hangar from stage 3 on (16 MB ROM, Expansion Pak required). As of 15 October 2026.

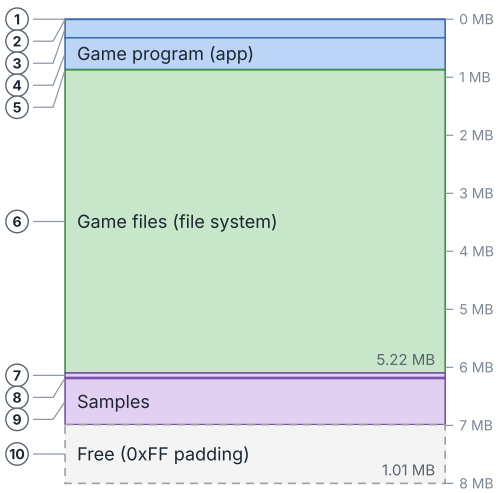
System / hardware · Game code · Game data · Music and sound · Heap (free working memory)
Fixed buffers (frame, depth, audio) · Unused (padding) · New in Hawk's Hangar

ROM (cartridge / .z64 file)

Drawn to scale: both bars use the same scale, so the 16 MB ROM is twice as tall. The numbers on the left are listed below each bar with ROM offsets and sizes.

Before: original ROM

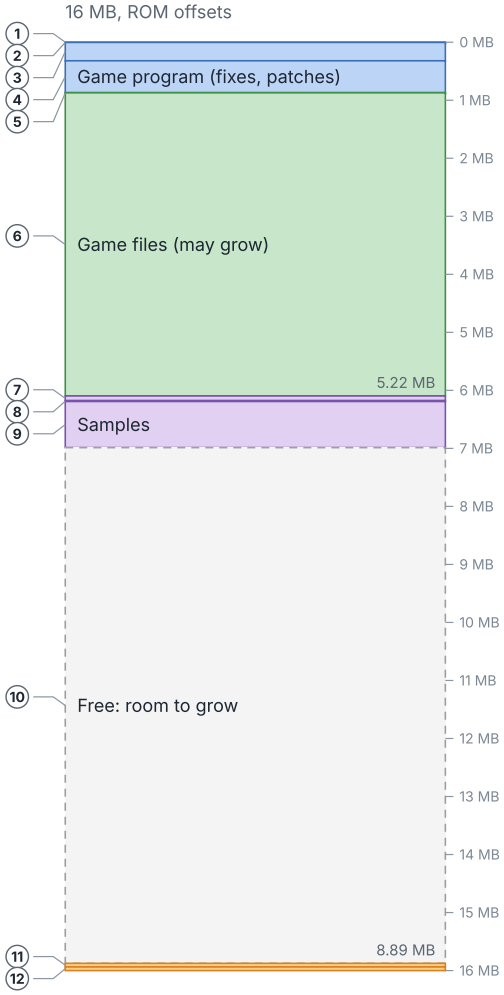
8 MB, ROM offsets



1	Header	000000–000040	64 B
2	IPL3 boot code	000040–001000	3.9 KB
3	Kernel + libultra + RSP microcode	001000–051E30	323.5 KB
4	Game program (app)	051E30–0DE720	562.2 KB
5	File table (UVRM)	0DE720–0DF5B0	3.6 KB
6	Game files (file system)	0DF5B0–618B70	5.22 MB
7	Sequences (music)	618B70–62D460	82.2 KB
8	Instrument bank	62D460–6314D0	16.1 KB
9	Samples	6314D0–6FCBE0	813.8 KB
10	Free (0xFF padding)	6FCBE0–800000	1.01 MB

After: hack ROM

16 MB, ROM offsets



1	Header (new checksum)	000000–000040	64 B
2	IPL3 boot code	000040–001000	3.9 KB
3	Kernel + loader (code cave)	001000–051E30	323.5 KB
4	Game program (fixes, patches)	051E30–0DE720	562.2 KB
5	File table (UVRM)	0DE720–0DF5B0	3.6 KB
6	Game files (may grow)	0DF5B0–618B70	5.22 MB
7	Sequences (music)	618B70–62D460	82.2 KB
8	Instrument bank	62D460–6314D0	16.1 KB
9	Samples	6314D0–6FCBE0	813.8 KB
10	Free: room to grow	6FCBE0–FE0000	8.89 MB
11	Data area image (mission table)	FE0000–FF0000	64.0 KB
12	Game module (own code)	FF0000–1000000	64.0 KB

ROM: the sections

Header	The first 64 bytes: the game's name ("Pilot Wings64"), game code NPWE and the boot checksum. The editor recomputes the checksum after every build, because code inside the checked first megabyte changes.
IPL3 boot code	Start-up code for the CIC chip (6102). It checks the first megabyte of the ROM and loads the start of the program. Unchanged.
Kernel + libultra + RSP microcode	The game's base system ("uv" functions: memory, files, graphics, sound), Nintendo's library libultra and the microcode of the graphics and sound processor. Copied to 0x80200000 at start-up. New: the "code cave" (ROM 0x2B4A4–0x2B548, a function that does nothing useful) holds the photo helpers and the loader of the game module; a few instructions of the memory manager are changed (code area, data area, text tables).
Game program (app)	The actual game logic: vehicles, missions, scoring, menus, HUD. Copied to 0x802CA900. Code fixes and game patches exchange single instructions here; the editor also writes tables (result rows, demo button).
File table (UVRM)	List of all game files with type and size. The game finds each file by adding up the sizes. The editor writes a new one when a file changes.
Game files	All content: island models, terrain, textures, missions (UPWT), texts, demo flights, camera paths and more, many of them packed (MIO0/GZIP). The project's changed files are put in here; the file system may grow.
Sequences, instrument bank, samples	Music in the Compact MIDI format, the instrument bank and the samples. Their addresses are fixed in the code; if the files no longer fit in front of them, the editor moves the whole block behind the files and updates the references.
Free	About 1 MB of padding (0xFF) in the original; about 8.9 MB in a hack ROM: room for larger files, new islands and missions. Empty space costs almost nothing in the BPS patch.
Data area image	64 KB in front of the module. It holds the mission table (header + 256 bytes per mission with settings: conditions, fuel, hard end time, more later). The module copies it into the data area in RAM at start-up.
Game module	The last 64 KB of the ROM: the editor's own code (module.c, about 18 KB). It checks for the Expansion Pak, applies conditions, hard end time and goal objects, shows messages and provides the debug output. It always sits at the end of the ROM, whatever its size.

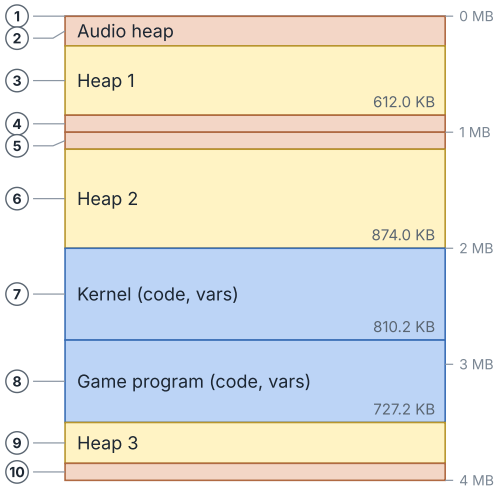
Every build starts again from the clean ROM: the editor puts in the project's files, places music, data area and module, applies the code fixes and game patches (single instructions in kernel and game program) and recomputes the boot checksum.

RAM (the console's working memory)

Drawn to scale (KSEG0 addresses from `0x80000000`): both bars use the same scale, so 8 MB is twice as tall as 4 MB. The game's memory manager has a list of protected blocks (buffers, program, audio); everything in between is heap. The editor puts new **data** in front of the program block (growing downwards) and new **code** behind the game's variables (growing upwards).

Before: without Expansion Pak

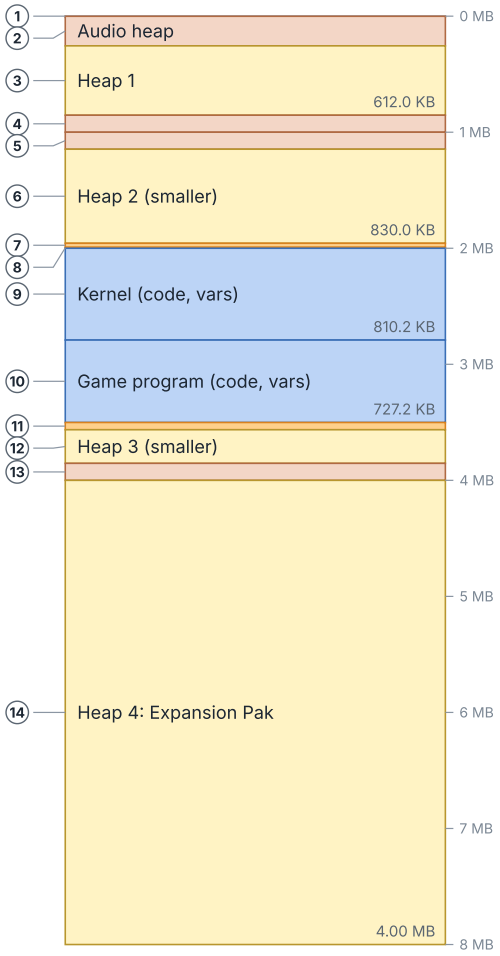
4 MB RDRAM, RAM addresses



1	Exception vectors, OS values	`80000000–80000400`	1.0 KB
2	Audio heap	`80000400–800417DC`	261.0 KB
3	Heap 1	`800417DC–800DA800`	612.0 KB
4	Frame buffer 1	`800DA800–80100000`	150.0 KB
5	Frame buffer 2	`80100000–80125800`	150.0 KB
6	Heap 2	`80125800–80200000`	874.0 KB
7	Kernel (code, vars)	`80200000–802CA900`	810.2 KB
8	Game program (code, vars)	`802CA900–803805E0`	727.2 KB
9	Heap 3	`803805E0–803DA800`	360.5 KB
10	Depth buffer (Z-buffer)	`803DA800–80400000`	150.0 KB

After: with Expansion Pak

8 MB RDRAM, RAM addresses



1	Exception vectors, OS values	`80000000–80000400`	1.0 KB
2	Audio heap	`80000400–800417DC`	261.0 KB
3	Heap 1	`800417DC–800DA800`	612.0 KB
4	Frame buffer 1	`800DA800–80100000`	150.0 KB
5	Frame buffer 2	`80100000–80125800`	150.0 KB
6	Heap 2 (smaller)	`80125800–801F5000`	830.0 KB
7	Data area: mission table	`801F5000–801FE000`	36.0 KB
8	Text tables (1024 texts)	`801FE000–80200000`	8.0 KB
9	Kernel (code, vars)	`80200000–802CA900`	810.2 KB
10	Game program (code, vars)	`802CA900–803805E0`	727.2 KB
11	Code area: game module	`803805E0–803905E0`	64.0 KB
12	Heap 3 (smaller)	`803905E0–803DA800`	296.5 KB
13	Depth buffer (Z-buffer)	`803DA800–80400000`	150.0 KB
14	Heap 4: Expansion Pak	`80400000–80800000`	4.00 MB

RAM: the sections

Exception vectors, OS values	Entry points for interrupts and values of the operating system, e.g. osMemSize (0x80000318: 4 or 8 MB) and the TV standard. The module reads osMemSize to see whether an Expansion Pak is present.
Audio heap	Fixed memory of the sound output (playback state, buffers).
Heap 1–3	Free working memory: when an island and a mission are loaded, everything comes from here – models, terrain, textures, mission objects, texts. Without the Expansion Pak about 1.8 MB in total, cleared at every level start. Without a Pak, part of heap 1 (from 0x80080000) holds the frame of the "EXPANSION PAK REQUIRED" screen.
Frame buffers 1 and 2	Two pictures of 320 × 240 pixels (16 bits): one is shown while the game draws the next one into the other.
Kernel	Code and variables of the base system (copied from the ROM). The memory manager protects it as the "program block", which now starts at 0x801F5000.
Game program	Code and variables of the game logic up to 0x803805E0. The old text tables in this area (0x80377F10–0x80378CCC) are unused since expanded-texts-v1.
Data area	New, 36 KB in front of the text tables, taken from heap 2: header + mission table (up to 128 missions, ends at 0x801FD100), the rest is free. New data tables grow downwards from here (e.g. a larger object memory for missions).
Text tables	Since the code fix expanded-texts-v1: names and pointers of up to 1024 texts (before: 439, inside the game program).
Code area: game module	64 KB right behind the game's variables, taken from heap 3. The loader copies the game module here before the game starts. New code grows upwards from here.
Depth buffer (Z-buffer)	Stores the distance of every pixel, so near things are drawn in front of far ones. Also used as scratch memory while loading.
Heap 4: Expansion Pak	The additional 4 MB. The game uses them as heap on its own (the heap ends at osMemSize): room for larger islands, more objects and new tables. Every hack requires the Pak.

Start-up (after)

1. The IPL3 boot code checks the first megabyte and starts the kernel, which copies itself to `0x80200000` and sets up the operating system and the video output.
2. The kernel copies the game program to `0x802CA900`. Instead of starting it right away, it calls the **loader in the code cave**.
3. The loader copies the game module from the end of the ROM to `0x803805E0` and starts it.
4. The module reads osMemSize. With 4 MB it shows "EXPANSION PAK REQUIRED" and stops. With 8 MB it copies the mission table into the data area (`0x801F5000`) and returns.
5. The game starts as usual. Its memory manager leaves the data area, the text tables and the code area alone and uses the additional 4 MB as heap.